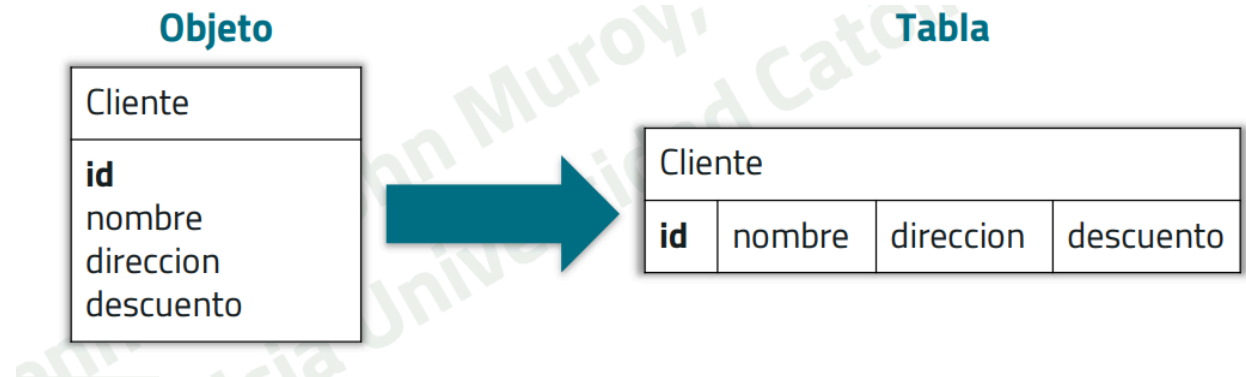


1. Patrones de Mapeo Básico

Identity Field



Definición: Almacena el ID del registro de BD en el objeto.

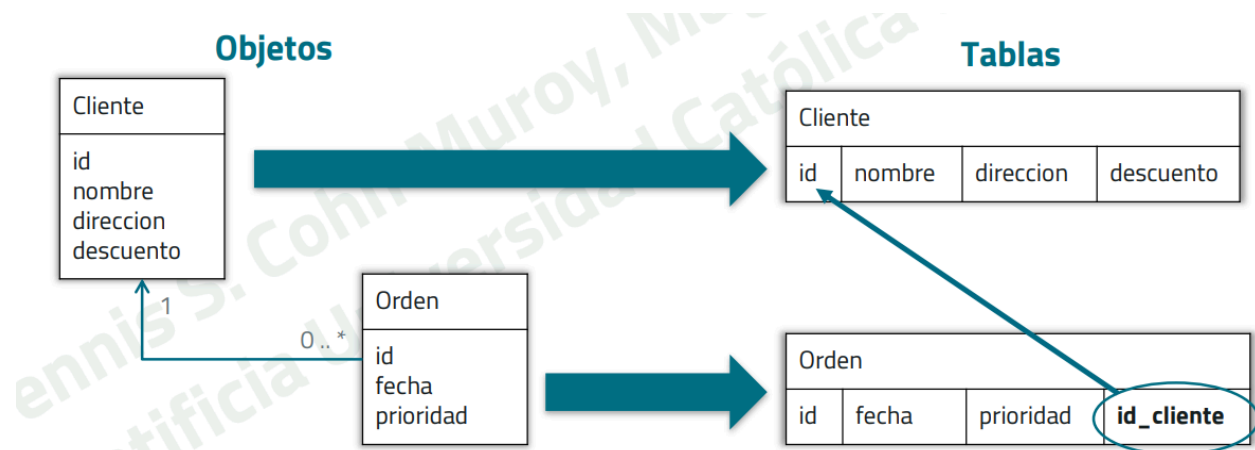
Características:

- Mapea objetos en memoria ↔ registros en BD
- El objeto contiene su identificador de base de datos

Estrategias de ID:

- Llave representativa: DNI, código (puede cambiar)
- Llave no representativa: Valor incremental, UUID (inmutable)
- Generación: Auto-increment, GUID, método personalizado

Foreign Key Mapping



Definición: Representa relaciones 1:1 o 1:N mediante llaves foráneas.

Ejemplo:

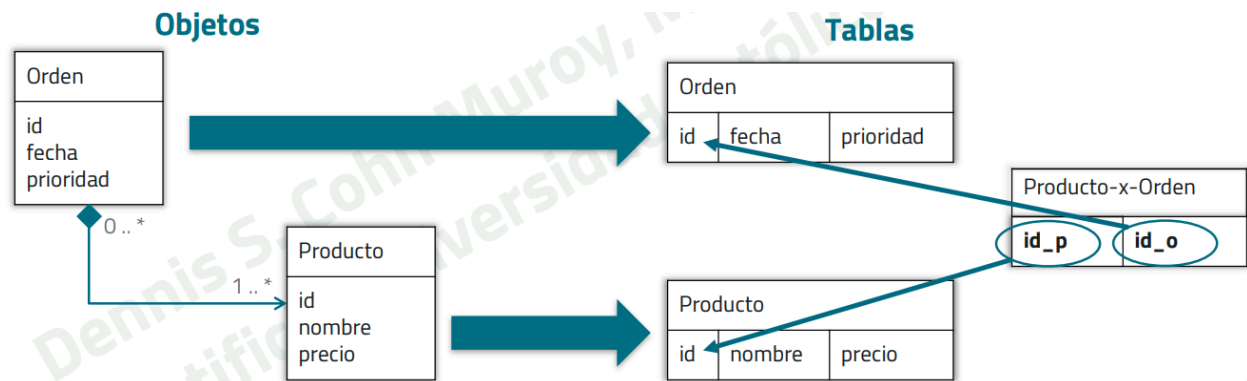
text

Cliente (1) ----- (N) Orden

Tablas:

- Cliente(id, nombre, ...)
- Orden(id, fecha, id_cliente)

Association Table Mapping



Definición: Representa relaciones N:M mediante tabla intermedia.

Ejemplo:

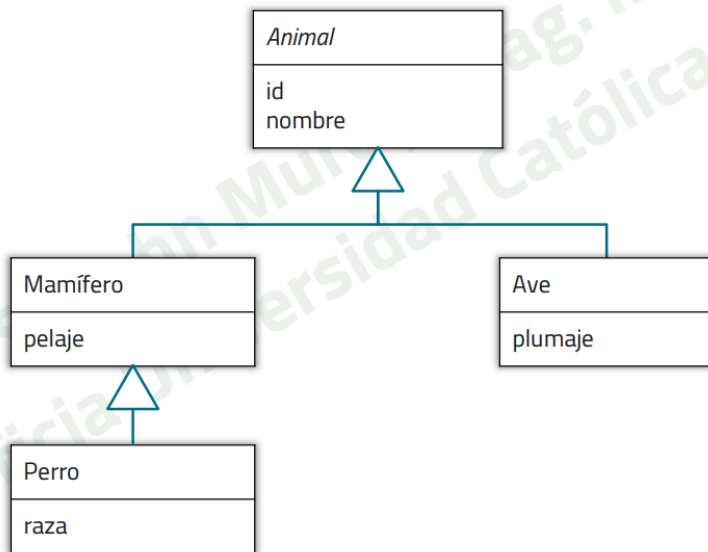
text

Orden (N) ----- (M) Producto

Tablas:

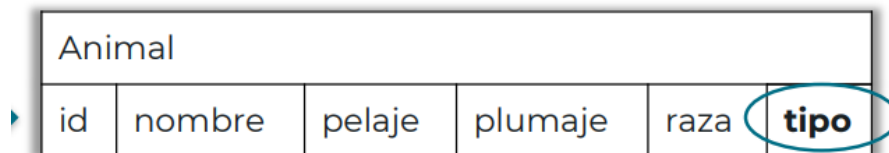
- Orden(id, fecha)
- Producto(id, nombre)
- Orden_Producto(id_orden, id_producto)
-

2. Patrones para Herencia 🎯



Single Table Inheritance

Definición: Todas las clases de la herencia en una sola tabla.



Ejemplo (Animales):

sql

```
Animal(id, nombre, tipo, pelaje, plumaje, raza)
```

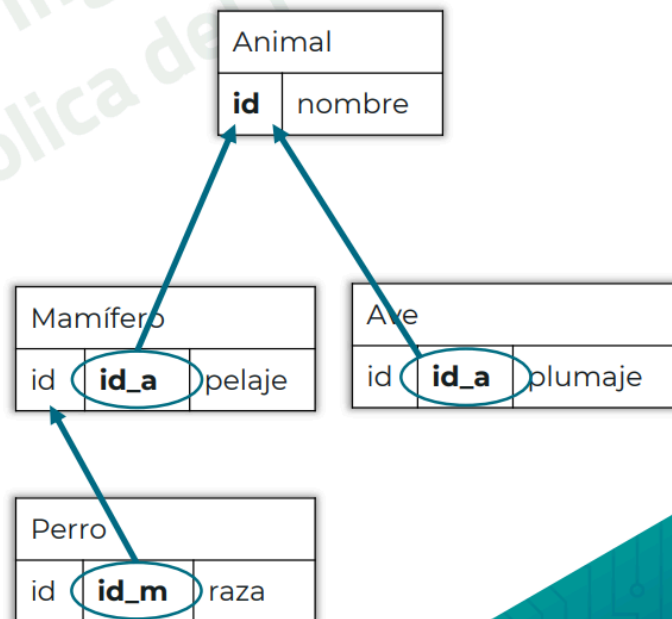
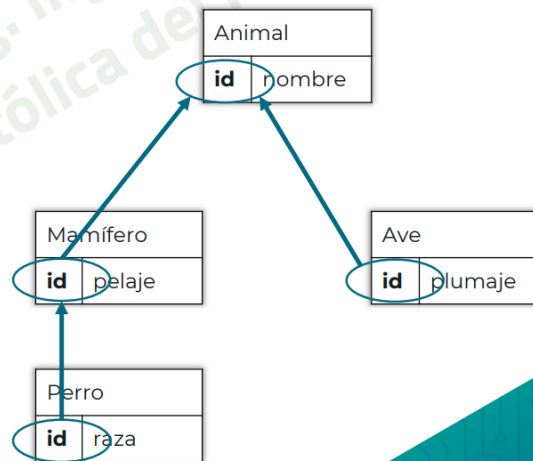
Ventajas:

- ✓ Sin JOINS - consultas rápidas
- ✓ Una tabla simple
- ✓ Fácil mover atributos entre clases

Desventajas:

- **×** Muchos NULLs - desperdicio de espacio
- **×** Tabla puede crecer demasiado
- **×** Complejo determinar importancia de columnas

Class Table Inheritance



Definición: Cada clase tiene su propia tabla.

Ejemplo (Animales):

```
sql
```

```
Animal(id, nombre)
```

```
Mamifero(id, pelaje)
```

```
Ave(id, plumaje)
```

```
Perro(id, raza)
```

Ventajas:

-  Sin desperdicio de espacio
-  Modelo entendible
-  Relación directa clase-tabla
-  Menor impacto al agregar clases

Desventajas:

-  Múltiples JOINS - consultas complejas
-  Mover atributos = cambiar esquema
-  Clase padre puede ser cuello de botella

Concrete Table Inheritance

Mamífero		
id	nombre	pelaje

Ave		
id	nombre	plumaje

Perro			
id	nombre	pelaje	raza

Definición: Solo las clases concretas tienen tabla, con todos los atributos.

Ejemplo (Animales):

```
sql
```

```
Mamifero(id, nombre, pelaje)
```

```
Ave(id, nombre, plumaje)
```

```
Perro(id, nombre, pelaje, raza)
```

Ventajas:

- ☒ Sin JOINS para clases concretas
- ☒ Sin desperdicio de espacio
- ☒ Tablas usadas solo cuando se necesita la clase
- Menor impacto al agregar clases hijas

Desventajas:

- ☒ Cambios en clase padre afectan múltiples tablas
 - ☒ Consultas complejas para clase padre
 - ☒ Duplicación de atributos comunes
 -
-

3. Guía de Selección para Herencia

Single Table Inheritance:

- ☒ Cuando las subclases tienen pocos atributos específicos
- ☒ Cuando el rendimiento en lectura es crítico
- ☒ Para jerarquías simples

Class Table Inheritance:

- ☒ PARA VEHÍCULOS (como en el caso anterior)
- ☒ Cuando se valora la normalización
- ☒ Para modelos complejos con mucha herencia

Concrete Table Inheritance:

- ☒ Cuando casi no se consulta por la clase padre
 - ☒ Cuando las subclases son muy diferentes
 - ☒ Para optimizar consultas específicas
-

4. Recomendación para el Caso de Vehículos

Class Table Inheritance es la mejor opción porque:

- Mantiene la normalización
- Permite expansión futura fácil
- Relación clara entre clases y tablas
- Minimiza desperdicio de espacio
- Aunque tiene JOINS, son manejables con índices apropiados

Estructura recomendada:

```
text
```

```
vehiculos(id, marca, modelo, año)
```

```
vehiculos_aereos(id, altura_maxima)
```

```
vehiculos_terrestres(id, numero_ruedas)
```

```
helicopteros(id, numero_motores, capacidad_pasajeros)
```

```
automoviles(id, numero_puertas, tipo_transmision)
```



Resumen Ejecutivo

Patrón	Uso Ideal	Complejidad	Rendimiento
Single Table	Herencias simples	Baja	✓ Lectura
Class Table	Herencia compleja	Media	⚠ Con JOINS
Concrete Table	Subclases muy distintas	Alta	✓ Por clase

Para sistemas con evolución futura: Class Table Inheritance ofrece el mejor balance entre mantenibilidad y rendimiento.